

Plano de Implementação — Componentes Animados

Enriquecimento visual de Apresentações e Landing Pages (14 componentes, v1+v2)

Auditoria técnica do pipeline

2026-08-05

Sumário

Plano de Implementação — Componentes Animados	1
Descoberta técnica que baseia o plano	1
Fase v1 — implementar agora (8 componentes)	3
Fase v2 — catalogar, implementar sob demanda (6 componentes)	5
Contratos de dados	5
Verificação de não-regressão	6
Aplicação imediata — Kit inLego	6

Plano de Implementação — Componentes Animados

Origem: análise do slide 5 de `output/kit-start-flex/apresentacao/index.html` (indicador de torque) — um componente animado de alta qualidade, criado ad hoc, nunca catalogado em `aplicador-marca-conexao/SKILL.md`. Este plano formaliza esse padrão e o estende para 14 componentes, cobrindo apresentação e landing-page (os únicos materiais em HTML vivo — arte-
* vira PNG estático, animação não sobrevive ao screenshot).

Descoberta técnica que baseia o plano

`templates/apresentacao.html` já embute genericamente o CSS do gauge e do fluxo (usado por **todo** projeto) — o que faltava era (a) parametrização real (hoje os rótulos “45 Ncm (Slim)”/“60 Ncm (NP)” estão **hardcoded no Python**, não vêm do dado real do projeto) e (b) um jeito explícito do redator-
* pedir um componente, em vez de depender de `compiler.html.py` adivinhar pelo título do slide (“torque” in `titulo.lower()`, “script”/“spin” in `titulo.lower()` — funciona, mas é frágil e não escala para 14 tipos).

Decisão de arquitetura: adicionar um campo opcional componente (apresentação, por slide) / enriquecimentos (landing-page, por seção) ao `schema/slides.json` / `conteudo.json`. Quando presente, o compilador usa esse dado explícito. Quando ausente, o comportamento **legado por palavra-chave continua exatamente igual** — refatorado para chamar as mesmas funções genéricas novas, com os mesmos valores hardcoded de antes (zero mudança de output em `kit-start-flex/kit-stop-drill`, só remoção de duplicação).

Animação parametrizada por instância usa **custom properties CSS** (`--gauge-offset`, `--barra-pct` etc.) setadas via `style=""` inline pelo Python, com a transição real definida uma vez na classe

— permite múltiplas instâncias do mesmo componente na mesma página, cada uma com seu próprio valor-alvo, sem gerar CSS duplicado por instância.

Fase v1 — implementar agora (8 componentes)

#	Componente	Gatilho de conteúdo	Técnica	Esforço
1	Gauge circular (medidor com faixa/limite)	Dado numérico com limite/faixa de segurança	SVG arco + stroke-dashoffset/rotate() via custom property (já existe, generalizar)	Baixo (refactor)
2	Fluxo em etapas numeradas (SPIN/processo)	Processo sequencial, script de vendas	.fluxo-passo + seta, animation-delay escalonado (já existe, generalizar)	Baixo (refactor)
3	Contador animado (count-up)	Estatística JS love (requestAnimationFrame) disparada no mesmo hook (ex.: “50% menos tempo”) que já ativa .ativo/.visível	Médio	
4	Rosca/donut percentual	“% do todo” (cobertura, redução, taxa)	SVG círculo + stroke-dasharray proporcional, mesma técnica do gauge	Baixo
5	Accordion nativo	Perguntas/respostas (objeções, dúvidas)	<details>/<summary> nativo — zero JS	Baixo
6	Comparativo de barras	Múltiplas especificações do mesmo tipo lado a lado	div com width/height animado via custom property	Baixo
7	Badge com pulso	Selo/certificação que merece destaque sutil	Variante CSS de .badge já existente + @keyframes pulse	Trivial
8	Divisor de seção animado	Transição decorativa entre seções longas (landing)	Linha SVG com var(--gradiente-assinatura) que se desenha ao entrar na viewport	Trivial

Fase v2 — catalogar, implementar sob demanda (6 componentes)

Documentados como padrão/template pronto em `aplicador-marca-conexao/SKILL.md`, **sem** função Python dedicada nesta rodada — motivo: exigem mais contexto de conteúdo específico por projeto (matriz depende de quantos sistemas comparar; diagrama técnico exige arte bespoke por produto) ou têm menor retorno imediato frente ao esforço de generalizar.

#	Componente	Por que fica para v2
9	Matriz/grade de compatibilidade	Nº de linhas/colunas varia muito por projeto — generalizar exige mais decisões de layout do que os demais
10	Comparativo antes/depois (wipe)	Precisa de 2 imagens ou 2 blocos de conteúdo espelhados — schema mais complexo
11	Timeline horizontal com traço progressivo	Sobreposição parcial com fluxo (v1) — vale medir demanda real antes de manter os dois
12	Diagrama técnico “desenhado” (line-draw)	Exige arte SVG bespoke por produto — não é um template genérico, é ilustração sob encomenda
13	Card com flip (frente/verso)	Precisa de “verso” com conteúdo real (REGRA 6) — nem todo componente do dossiê tem informação suficiente para as duas faces
14	Cápsula/pílula de nível (fill)	Sobreposição funcional com gauge/donut — mesmo dado, forma alternativa

Contratos de dados

`slides.json` (apresentação) — novo campo opcional por slide:

```
{
  "tipo": "conteudo",
  "titulo": "Tabela de Torques e Limites",
  "corpo": [...],           // mantido como fallback/legado
  "componente": {
    "tipo": "gauge",        // gauge | fluxo | contador | donut | accordion |
    barras
    "dados": { "valor": 60, "min": 0, "max": 80, "unidade": "Ncm",
```

```

        "titulo_indicador": "Indicador de Torque Seguro",
        "marcas": [{ "valor": 45, "label": "45 Ncm (Slim)"}] }
    }
}

```

conteudo.json (landing-page) — novo array opcional de topo:

```

{
  "enriquecimentos": [
    { "secao": "destaques", "tipo": "donut", "dados": { "percentual": 50,
"label": "Redução no tempo de cadeira" } },
    { "secao": "prova", "tipo": "accordion", "dados": { "itens": [{"pergunta":
"...", "resposta": "..."}] } }
  ]
}

```

Verificação de não-regressão

Antes de aplicar a qualquer material novo: `auditar-projeto.py kit-start-flex --estrito` e `auditar-projeto.py kit-stop-drill --estrito` devem continuar CONFORME após o refactor — o caminho por palavra-chave (torque/script/spin/objeções) precisa produzir HTML equivalente ao atual, só reescrito para reusar as novas funções genéricas.

Aplicação imediata — Kit inLego

Conteúdo do dossiê que justifica enriquecimento real (nunca forçar componente sem dado): -

Fluxo: script SPIN já previsto no `brief_criativo.json` → componente: `fluxo`. - **Accordion:** 3 pares objeção/resposta → substitui tabela plana por componente: `accordion`. - **Donut:** “reduz o tempo de cadeira pela metade” (50%) → componente: `donut` na `landing-page`.

Gauge/barras **não** entram no Kit inLego — o dossiê não tem dado de torque/faixa numérica comparável; forçar um desses violaria REGRA 6 (nunca inventar dado).